Rajalakshmi Engineering College

Name: SEJAL SAI S
Email: 240701482@rajalakshmi.edu.in
Roll no: 240701482
Phone: 9361762052
Branch: REC
Department: I CSE FE
Batch: 2028
Degree: B.E - CSE

Scan to verify results

NeoColab_REC_CS23231_DATA STRUCTURES

REC_DS using C_Week 1_COD_Question 5

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Imagine you are tasked with developing a simple GPA management system using a singly linked list. The system allows users to input student GPA values, insertion should happen at the front of the linked list, delete record by position, and display the updated list of student GPAs.

Input Format

The first line of input contains an integer n, representing the number of students.

The next n lines contain a single floating-point value representing the GPA of each student.

The last line contains an integer position, indicating the position at which a student record should be deleted. Position starts from 1.

Output Format

After deleting the data in the given position, display the output in the format "GPA: " followed by the GPA value, rounded off to one decimal place.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
3.8
3.2
3.5
4.1
2
Output: GPA: 4.1
GPA: 3.2
GPA: 3.8

Answer

```
#include<stdio.h>
#include<stdlib.h>
struct node{
   float gpa;
   struct node *Next;
};
typedef struct node Node;
void insertbeg(Node* List,float);
void deletemid(Node *List,int );
void traverse(Node* List);
Node *Find(Node* List,int );
void freeList(Node* List);
int main()
{
   Node *List=(Node*)malloc(sizeof(Node));
   List->Next=NULL;
   int n,p;
   float e;
   if(scanf("%d",&n)!=1){
      return 1;
```

```
    }
    for(int i=0;i<n;i++){
        if(scanf("%f",&e)!=1)return 1;
        insertbeg(List,e);
    }
    if(scanf("%d",&p)!=1)return 1;
    deletemid(List,p);
    traverse(List);
    freeList(List);
    return 0;
}
Node *Find(Node *List,int p){
        if(p<=0)return List;
        Node *position=List;
        int i=0;
        while(position!=NULL && i<p-1){
            position=position->Next;
            i++;
        }
        return position;
}

void insertbeg(Node* List,float e){
        Node *newnode=(Node*)malloc(sizeof(Node));
        newnode->gpa=e;
        if(List->Next==NULL){
            newnode->Next=NULL;
        }
        else{
            newnode->Next=List->Next;
        }
        List->Next=newnode;
}

void deletemid(Node * List,int p){

        Node *position=Find(List,p);
        Node *TempNode=position->Next;
        position->Next=TempNode->Next;
        free(TempNode);
}
```

```
void traverse(Node *List){
    Node *temp=List;
    while(temp->Next!=NULL){
        temp=temp->Next;
        printf("GPA: %.1f\n",temp->gpa);
    }
}
void freeList(Node *List){
    Node*temp;
    while(List!=NULL){
        temp=List;
        List=List->Next;
        free(temp);
}
}
```

Status : Correct **Marks : 10/10**